Gestão Financeira e Controle de Frota

Relatório Executivo

Disciplina: [PREENCHER]
Professor(a): [PREENCHER]
Integrantes: Thainá Mariano · Hygor · Paulo Henrique · Teodoro Kauan · [CONFERIR nomes completos e RA]
Repositório: https://github.com/MarianoThaina/GestaoFrota
Vídeo de demonstração: [PREENCHER link]
Data: [PREENCHER] de novembro de 2026

Rascunho de 04/11/2026. Montado a partir do código e do histórico do repositório. Os trechos entre colchetes, como [PREENCHER] e [CONFIRMAR], dependem do grupo. As seções 6 e 7 também dependem da hospedagem escolhida e do roadmap original do documento do projeto.

1. Resumo executivo

O **Gestão de Frota** é um sistema para pequenas transportadoras controlarem, num lugar só, o dinheiro da empresa e o uso dos veículos. É composto por um **aplicativo Android** (.NET MAUI), uma **API REST** (ASP.NET Core), um **banco SQL Server** (EF Core) e **páginas web** de cadastro.

O sistema cobre três frentes:

- **Financeiro:** entradas e saídas com comprovante fotografado, dívidas e financiamentos com parcelas geradas automaticamente, saldo atual e projeção de caixa mês a mês.
- **Frota:** cadastro de veículos com validação de placa (antiga e Mercosul), rotas, viagens com controle de hodômetro (que funcionam **sem internet** no celular do motorista) e manutenções que geram a despesa sozinhas.
- **Relatórios:** indicadores, gráficos interativos de fluxo de caixa e de despesas por categoria, rentabilidade e custo por quilômetro de cada rota e veículo, ranking de eficiência e exportação do período em PDF e CSV.

O acesso é feito com a **conta Google** (sem senha própria) e cada um dos quatro perfis (Administrador, Gestor de Frota, Financeiro e Motorista) vê só o que lhe cabe. A permissão é conferida também no servidor, não só escondida na tela.

Situação em 04/11/2026: das 19 user stories, 17 estão implementadas, a US14 (PDF) está em revisão no PR #18 e a US16 está parcial (ver a matriz na seção 3). Faltam, para a entrega, configurar o login Google real, hospedar a API e validar o `.apk` em celulares reais (seção 9).

2. Detalhamento dos módulos

2.1 Acesso e perfis (US15 a US18)

Funcionalidade	Como funciona
Login com Google	O app abre o login do Google e recebe um `id_token`. A API confere esse token, cria ou atualiza o usuário e devolve um JWT próprio. O app nunca vê a senha, e a sessão fica salva no `SecureStorage` do aparelho.
Perfis	Quatro perfis: Administrador, Gestor de Frota, Financeiro e Operador/Motorista. Uma conta nova entra como Motorista, e só o Administrador promove.
Menu por perfil	Menu lateral (Flyout) montado conforme o perfil. O Motorista vê só Viagens e Frota (consulta) e abre direto em Viagens.
Segurança no servidor	Todo endpoint tem `[Authorize(Roles = …)]`. O Motorista só recebe as próprias viagens, por um filtro na consulta da API.

2.2 Financeiro (US01 a US05)

Funcionalidade	Como funciona
Categorias e formas de pagamento	Cadastro com exclusão lógica: desativar não apaga o histórico que aponta para elas. Seis categorias e quatro formas vêm pré-cadastradas.

Funcionalidade	Como funciona
Movimentações	Entrada ou saída com valor, descrição, categoria, forma de pagamento e vencimento. Pode ficar Pendente ou Pago/Recebido, e Pago exige data de pagamento (validado no app, na API e no banco). Aceita comprovante pela câmera ou pela galeria, e o estorno mantém o registro.
Dívidas e financiamentos	Informados o valor da parcela, a quantidade e o 1º vencimento, a API gera as N parcelas pendentes numa única transação. Se uma falhar, nenhuma é gravada.
Saldo e projeção	O saldo atual soma só o que já foi pago ou recebido. A projeção soma, mês a mês, o saldo mais os pendentes pelo vencimento. Estornadas nunca entram.

2.3 Frota (US06 a US10)

Funcionalidade	Como funciona
Veículos	Placa antiga ou Mercosul, validada e guardada sem hífen, para a regra de placa única não ser burlada. Também guarda a capacidade de carga e a situação (Disponível, Em viagem, Em manutenção).
Manutenção	Ao registrar uma manutenção, a despesa é lançada no financeiro e o veículo passa a "Em manutenção", tudo na mesma transação. Ao concluir, o veículo volta a "Disponível".
Rotas	Nome, origem, destino e distância estimada, com desativação lógica.
Viagens	Abertura com hodômetro inicial. O encerramento exige que o hodômetro final seja maior que o inicial, e a quilometragem rodada fica gravada. **Abrir e encerrar funcionam offline:** o registro espera no celular e é reenviado quando a conexão volta.
Vínculo com o financeiro	Uma movimentação pode apontar para a viagem e, se for combustível, para a quantidade de litros. É isso que alimenta as estatísticas.

2.4 Relatórios (US10 a US14)

Funcionalidade	Como funciona
Indicadores	Resultado do período, entradas, saídas, a receber e a pagar no mês, pendentes em atraso, maior despesa e variação de saídas. O Início mostra ainda os veículos em viagem.
Gráficos	Colunas de entradas × saídas por mês e rosca de despesas por categoria, desenhados com o `GraphicsView` do próprio MAUI. Tocar num ponto mostra o valor exato.
Filtro de período	Atalhos de 3, 6, 12 e 24 meses ou tudo, além de mês/ano, ano inteiro e intervalo livre.
Frota no período	Rentabilidade, custo/km e km/l por rota e por veículo. O ranking mostra as 3 rotas mais lucrativas e os 3 veículos com menor custo/km.
Exportação	O período filtrado sai em **PDF** (cabeçalho, resumo e tabela, gerado na API com QuestPDF) ou em **CSV** para planilha. Os dois saem da mesma consulta, então os números batem.

2.5 Site web de cadastro

Páginas em HTML e JavaScript puro (`GestaoFrota.Web`) para Categorias, Formas de Pagamento e Usuários, consumindo a mesma API. Têm um modo demonstração, com dados em memória, que deve ser desligado na entrega.

3. Matriz de rastreabilidade

Cada user story ligada ao endpoint da API, à tela do app, ao teste automatizado e ao pull request que a entregou. Os testes cobrem as regras puras do domínio, que não dependem de banco nem de tela.

US	Requisito	API	Tela do app	Teste automatizado	PR	Situação
US01	Categorias	`CategoriasController`	`CategoriasPage`	—	commit de 23/08	Implementada
US02	Formas de pagamento	`FormasPagamentoController`	`FormasPagamentoPage`	—	commit de 23/08	Implementada (o app lista e cria)
US03	Movimentações e comprovante	`MovimentacoesController`	`MovimentacoesPage`, `MovimentacaoFormPage`, `ComprovantePage`	—	#11	Implementada
US04	Dívidas e parcelamento	`DividasController`	`DividasPage`, `DividaFormPage`, `DividaDetalhePage`	`CalculadoraFinanceiraTests`	#11	Implementada (quitação antecipada só registrada)
US05	Saldo e projeção	`FinanceiroController`	`DashboardPage`	`CalculadoraFinanceiraTests`	#11	Implementada

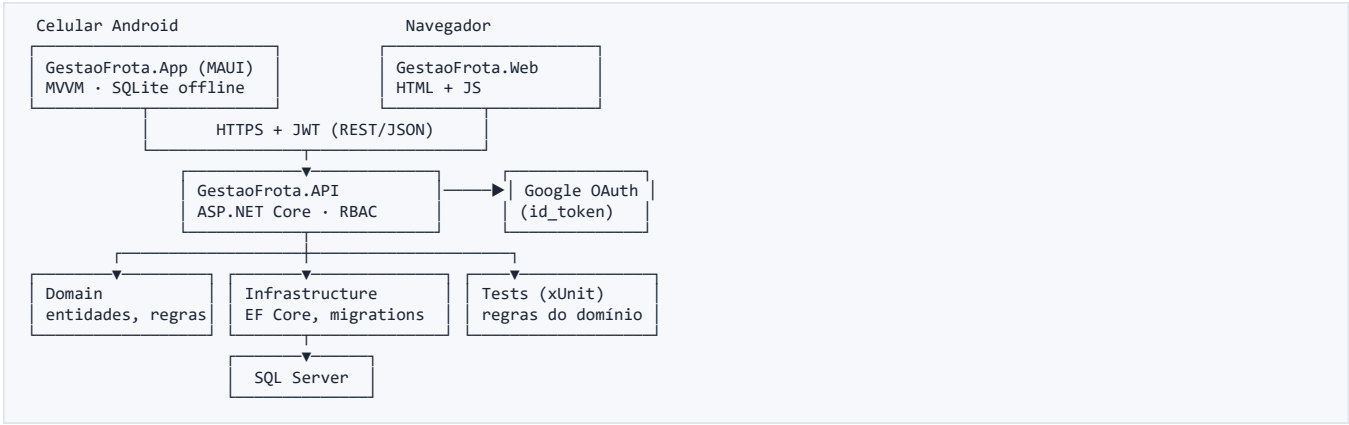
US	Requisito	API	Tela do app	Teste automatizado	PR	Situação
US06	Cadastro de frota	`VeiculosController`	`FrotaPage`, `VeiculoFormPage`	`ValidadorPlacaTests`	#14	Implementada
US06-b	Manutenção	`ManutencoesController`	`ManutencaoFormPage`	—	#14	Implementada
US07	Rotas	`RotasController`	`RotasPage`	—	#14	Implementada
US08	Viagens e hodômetro (offline)	`ViagensController`	`ViagensPage`, `ViagemFormPage`, `EncerrarViagemPage`	`RegrasViagemTests`	#14	Implementada
US09	Vínculo financeiro com viagem	`MovimentacoesController` (`viagemId`, `quantidadeLitros`)	`MovimentacaoFormPage`	—	#14	Implementada
US10	Estatísticas de rota e veículo	`RelatoriosController` (`/frota`)	`RelatoriosPage`	`CalculadoraFrotaTests`	#14	Implementada
US11	Painel de indicadores	`RelatoriosController` (`/resumo`)	`RelatoriosPage`, `DashboardPage`	`CalculadoraRelatoriosTests`	#13, #16	Implementada
US12	Gráficos de fluxo de caixa	`RelatoriosController` (`/resumo`)	`RelatoriosPage`	`CalculadoraRelatoriosTests`	#13, #16	Implementada
US13	Ranking de eficiência	`RelatoriosController` (`/frota`)	`RelatoriosPage`	`CalculadoraFrotaTests`	#14	Implementada
US14	Exportação (PDF e CSV)	`RelatoriosController` (`/exportar`)	`RelatoriosPage`	—	#13, #18	CSV na main, PDF em revisão (#18)
US15	Login com Google	`AuthController`	`LoginPage`	—	commit de 23/08	Implementada; falta o Client ID real
US16	Perfis e aprovação	`UsuariosController`	`UsuariosPage`	—	#4, #17 (revertido)	Parcial: perfis prontos; aprovação de cadastro revertida em 03/11 [CONFIRMAR decisão do grupo]
US17	Navegação por perfil	—	`AppShell`	—	#11, #14	Implementada
US18	Segurança de rotas	`[Authorize]` em todos os controllers	—	—	todos	Implementada

Requisitos não funcionais

Requisito	Onde é atendido
Offline-first	Cache SQLite de categorias, formas, histórico, veículos e rotas. Fila de reenvio para viagens, com IDs gerados no aparelho e reenvio idempotente.
Integridade do histórico	Exclusão lógica; `Restrict` nas chaves estrangeiras; check constraints de valor positivo, de “pago exige data” e de “hodômetro final maior que o inicial”.
Segurança	Login delegado ao Google, JWT, RBAC no servidor e nenhum segredo versionado (`user-secrets` ou variáveis de ambiente).
Testabilidade	48 testes xUnit sobre as cinco classes de regra pura do domínio.

4. Arquitetura

A solução segue uma arquitetura em camadas. O domínio não conhece o banco, a API não conhece o app, e o app só conversa com a API por HTTP.



Projeto	Responsabilidade
`GestaoFrota.Domain`	Entidades, enums e as regras puras: `CalculadoraFinanceira`, `CalculadoraRelatorios`, `CalculadoraFrota`, `ValidadorPlaca` e `RegrasViagem`.
`GestaoFrota.Infrastructure`	`AppDbContext`, mapeamentos do EF Core e migrations.
`GestaoFrota.API`	13 controllers REST, autenticação JWT, autorização por perfil, Swagger e geração do PDF.
`GestaoFrota.App`	App .NET MAUI para Android: 19 telas, MVVM com CommunityToolkit, cache SQLite e fila offline.
`GestaoFrota.Web`	Páginas web de cadastro.
`GestaoFrota.Tests`	Testes xUnit (referenciam só o Domain).

Decisões que mais pesaram

- **Regras de cálculo no domínio, sem banco:** podem ser testadas isoladamente e reaproveitadas pela API.
- **Gráficos sem biblioteca externa:** são desenhados com o `GraphicsView` do MAUI, o que mantém o app sem dependências novas.
- **PDF gerado na API:** evita uma biblioteca de PDF por plataforma no app. O PDF sai da mesma consulta do CSV, então os dois formatos sempre batem.
- **Comprovante guardado no banco** (`varbinary(max)`): não se perde em hospedagens com disco temporário.

5. Persistência e sincronização

Banco central. SQL Server com o schema versionado pelas migrations do EF Core:

Migration	Data	O que criou
`InitialCreate`	23/08/2026	Usuários, categorias, formas de pagamento, movimentações, dívidas, veículos, rotas, viagens e fretes
`FinanceiroCore`	11/10/2026	Status, vencimento, data de pagamento e comprovante da movimentação; valor da parcela, 1º vencimento e quitação antecipada da dívida
`ModuloFrota`	21/10/2026	Capacidade de carga, km rodada gravada, litros de combustível, tabela de manutenções

Um único comando cria o banco inteiro e semeia as categorias e as formas de pagamento padrão: `dotnet ef database update`.

Banco local (celular). SQLite com o cache do que o usuário precisa consultar sem internet: categorias, formas de pagamento, histórico de movimentações, veículos, rotas e viagens.

Sincronização.

1. Todo registro tem um **identificador `Guid` gerado no próprio aparelho**. Dois motoristas sem rede nunca geram o mesmo ID, e não há colisão ao sincronizar.
2. A viagem aberta ou encerrada sem conexão fica no SQLite marcada como pendente.
3. Ao abrir a tela de Viagens com conexão, o app reenvia os pendentes.
4. A API trata o reenvio como **idempotente**: o mesmo ID devolve a viagem já existente, e o mesmo hodômetro final devolve sucesso. Uma conexão que cai no meio do envio, portanto, não duplica nada.

As operações que mexem no caixa (lançar, pagar, estornar, cadastrar dívida) **exigem conexão** de propósito. O app avisa o usuário em vez de fingir que salvou, porque um saldo calculado em cima de dados não sincronizados levaria a decisões erradas.

6. Infraestrutura e custos

> [PREENCHER quando a hospedagem for escolhida] A tabela abaixo é a proposta. Os valores foram conferidos nas páginas dos provedores em 04/11/2026 e podem mudar.

Componente	Proposta	Custo
Repositório e versionamento	GitHub (repositório público)	R$ 0
Login	Google Identity (OAuth 2.0)	R$ 0
API	Azure App Service, plano Free F1, endereço `*.azurewebsites.net` com HTTPS	R$ 0 [CONFIRMAR o limite de CPU diário do F1]
Banco de dados	Azure SQL Database, oferta gratuita: 100.000 vCore-segundos e 32 GB por mês, renovados todo mês; pausa sozinho se estourar	R$ 0
Distribuição do app	`.apk` instalado direto no celular, sem Play Store	R$ 0 (a Play Store cobraria taxa única de cadastro)
Ferramentas	.NET SDK, Visual Studio/VS Code, Android SDK	R$ 0
Total mensal		**R$ 0**

Limites conhecidos da proposta:

- O plano F1 não aceita domínio próprio com certificado; o endereço padrão `azurewebsites.net` atende ao projeto.
- A oferta gratuita do Azure SQL não é compatível com a assinatura *Azure for Students Starter*. Se o grupo usar a conta de estudante, é preciso conferir qual oferta ela é antes de criar o banco.

Fontes: Azure SQL Database free offer · Azure App Service – certificados e domínio.

7. Cronograma: planejado × executado

> O planejado vem do roadmap de sprints quinzenais do documento do projeto, como citado no levantamento de 20/10/2026. [CONFERIR as datas e o conteúdo das Sprints 3 e 4 no documento original.] O executado vem das datas de merge no GitHub.

Sprint	Planejado	Executado	Desvio
1–2	Arquitetura em camadas, entidades, banco, login Google, perfis	16/08 a 23/08: estrutura da solução, EF Core, login Google, RBAC, CRUD de categorias e formas	No prazo
3–4	[CONFERIR] Movimentações, dívidas, saldo e projeção	31/08 a 30/09: protótipo navegável, gestão de usuários, site web, modelo do banco. **11/10:** núcleo financeiro completo (#11)	[CONFERIR com o roadmap] Núcleo financeiro entrou em 11/10
5 (a partir de 16/10)	Frota e viagens	21/10 a 24/10: módulo de frota inteiro, com offline (#14)	Adiantado: entregou também a Sprint 6
6 (até 30/10)	Vínculo com viagem, estatísticas (US09, US10)	24/10, junto com a Sprint 5 (#14)	Adiantado
7 (até 13/11)	Filtros de data, gráficos interativos, ranking, card de veículos	Relatórios em 20/10 (#13), filtros e toque em 02/11 (#16), ranking em 24/10 (#14)	Adiantado
8 (até 27/11)	Fila offline, aprovação de usuários, menu do Motorista, configuração do Google	Menu do Motorista e fila de viagens em 24/10 (#14); PDF (#18) e preparação do `.apk` (#20) em revisão; aprovação revertida	Em andamento

8. Dificuldades e lições aprendidas

Dificuldade	O que aprendemos
Parte do código foi escrita em máquinas sem o SDK .NET, então só compilava na máquina de outro integrante.	Separamos o roteiro de teste de cada PR (`docs/projeto/PR-*.md`) e combinamos que quem mescla compila antes. Um pipeline de CI no GitHub teria evitado essa dependência.
Migrations escritas à mão a partir do snapshot.	Funciona, mas é frágil. A conferência recomendada é gerar uma migration de teste depois: se o snapshot está certo, ela sai vazia.
Script SQL inicial em PostgreSQL, com o projeto em SQL Server.	Uma fonte só para o schema: as migrations. O script foi removido para não confundir.

Dificuldade	O que aprendemos
Conflitos recorrentes no mesmo arquivo de documentação (`estado-atual.md`) quando vários PRs ficavam abertos ao mesmo tempo.	PRs menores e mesclados em ordem, e cada PR acrescenta a sua própria seção em vez de editar as dos outros.
A receita de frete podia ser contada duas vezes (entidade `Frete` × movimentação vinculada à viagem).	O cálculo de rentabilidade usa uma fonte só, as movimentações vinculadas, e ignora o `Frete.ValorFrete` . [CONFIRMAR o papel final desse campo]
Um arquivo de configuração local (`appsettings.Development.json`) chegou a ser versionado no início.	Foi removido em 14/09 e entrou no `.gitignore` . Desde então, segredo fica só em `user-secrets` ou variável de ambiente, e o repositório público guarda apenas placeholders.
Aprovação de usuários (US16) mesclada e revertida em seguida.	[PREENCHER com o motivo do grupo]

9. Pendências para a entrega final

1. Criar o Client ID no Google Cloud e preenchê-lo no app, na API e no site.
2. Hospedar a API (seção 6) e apontar o app para ela (`AppConfig.ApiPublica`).
3. Rodar `dotnet build` e `dotnet test` , gerar o `.apk` e testar em celulares Android reais.
4. Mesclar os PRs em revisão: #18 (PDF) e #20 (`.apk` e documentação).
5. Decidir a US16 (aprovação de cadastro) após o revert de 03/11.
6. Capturas de tela, manual do usuário e vídeo de 3 a 5 minutos.

Anexo: telas do aplicativo

[INSERIR as capturas de tela do app rodando no celular: login, menu por perfil, movimentações, dívida com parcelas, frota, viagem offline, relatórios com gráficos e o PDF exportado.]